Project Report: Edge-AI Fall Detection System with Localized IoT Actuation

1. Project Overview

The objective of this project was to develop a real-time, privacy-first Fall Detection system that operates entirely on edge hardware. By utilizing a smartphone's built-in IMU (Inertial Measurement Unit) sensors, the system continuously monitors user movement and processes the data through a localized Deep Learning model. Upon detecting a fall, the mobile device dispatches an unencrypted HTTP POST payload to a custom ESP32 microcontroller over a localized, internet-independent Wi-Fi network, triggering an immediate physical alarm.

2. Data Collection Methodology

To train a highly personalized model, custom telemetry data was recorded using a mobile Sensor Logger application.

- **Sensors Logged:** 3-Axis Accelerometer and 3-Axis Gyroscope.
- **Sampling Rate:** Strictly enforced at 50Hz (one reading every 20ms) to capture the rapid micro-movements of a fall.
- **Labeling Strategy:**
    - **Class 0 (ADL - Activities of Daily Living):** Recorded standard non-fall movements including walking, sitting, standing, and light jumping.
    - **Class 1 (Falls):** Recorded simulated forward and backward falls onto a mattress to safely capture the gravitational spikes associated with human collapse.
- **Device Placement:** The device was kept in the user's front pocket in a fixed orientation during all recordings to ensure consistency.

3. AI Model Architecture

Instead of flattening all sensor data into a single vector, the system utilizes a **Dual-Stream 1D Convolutional Neural Network (CNN)**.

1. **Stream A:** Processes the Accelerometer matrix (128 rows x 3 columns).
2. **Stream B:** Processes the Gyroscope matrix (128 rows x 3 columns).
3. **Fusion:** Features are extracted independently to learn the spatial relationships of acceleration and rotation, then fused in a Fully Connected layer to output a final probability score (0.0 to 1.0).

The model was converted to a highly compressed .tflite format, allowing it to be bundled directly inside the mobile application for zero-latency inference.

4. Mobile Application Architecture

The deployment environment was built using Native Android (Kotlin) to ensure robust, low-level access to the SensorManager without the UI thread bottlenecking the data stream.

The application consists of four modules:

1. **Sensor Listener:** Subscribes to the hardware at SENSOR_DELAY_GAME (approx. 50Hz).
2. **Dual-FIFO Buffer:** Maintains a rolling window of the last 128 timesteps (2.56 seconds of memory).
3. **Inference Engine:** Feeds the buffer to the .tflite model and extracts the confidence score.
4. **Alert Dispatcher:** Overrides Android's usesCleartextTraffic security policy to allow local, unencrypted network communication. Upon crossing the AI threshold, it executes an asynchronous HTTP POST request to the local ESP32 server.

5. IoT Hardware Integration (ESP32 Actuator)

To provide a physical alert mechanism without relying on external cloud infrastructure, an ESP32 microcontroller (ASC-CSE IoT 2.0 Board) was integrated as the actuation layer.

- **Network Topology (SoftAP Mode):** To bypass modern smartphone security restrictions (like WPA3 protocols and cellular data routing interference), the ESP32 was configured in **Access Point (AP) Mode**. The ESP32 broadcasts its own localized Wi-Fi network (ESP32_Fall_Alarm) and assigns itself a static IP (192.168.4.1). The Android device connects directly to this closed-loop network.
- **HTTP Web Server:** The ESP32 hosts a lightweight web server on Port 80, listening specifically at the /trigger_alarm endpoint.
- **Non-Blocking Actuation:** Upon receiving the POST request, the ESP32 triggers an onboard SMD LED (Pin 33 / D8). To prevent hardware watchdog resets and maintain network stability, the system utilizes a non-blocking millis() stopwatch timer to keep the LED illuminated for exactly 5,000ms before resetting, rather than relying on thread-freezing delay() functions.

6. Real-World End-to-End Testing & Logcat Analysis

During live edge-testing, the system successfully ingested data, ran inferences continuously, and successfully bridged the software-to-hardware gap.

Calibration & Thresholding:

A calibration difference was observed between the raw Python training data and the Android

hardware output.

- **Resting/ADL State:** Current AI Score: 4.815E-4 (0.04% confidence)
- **Simulated Fall Event:** Current AI Score: 0.2934 (29.34% confidence)

Because the model was heavily compressed for the edge, the absolute confidence output was scaled down. However, the *relative* spike was massive (a 70,000% increase in confidence). Based on this empirical data, the deployment threshold was adjusted to a practical 0.20f.

Hardware Trigger Logs:

Once the threshold is crossed, the end-to-end trigger fires in milliseconds, as verified by the ESP32 Serial output:

```
Plaintext
```

- --- Starting ESP32 Hotspot ---
- ESP32 IP Address: 192.168.4.1
- HTTP server started!
-
- 🚨 EMERGENCY: FALL SIGNAL RECEIVED FROM PHONE! 🚨
- Alarm reset. LED OFF. Listening...
- 🚨 EMERGENCY: FALL SIGNAL RECEIVED FROM PHONE! 🚨
- Alarm reset. LED OFF. Listening...

7. Error Analysis: The "Table Drop" False Positive

Currently, the system successfully detects falls, but it triggers a false positive when the phone is taken out of the pocket and placed forcefully onto a table. There are two primary Data Science reasons for this behavior:

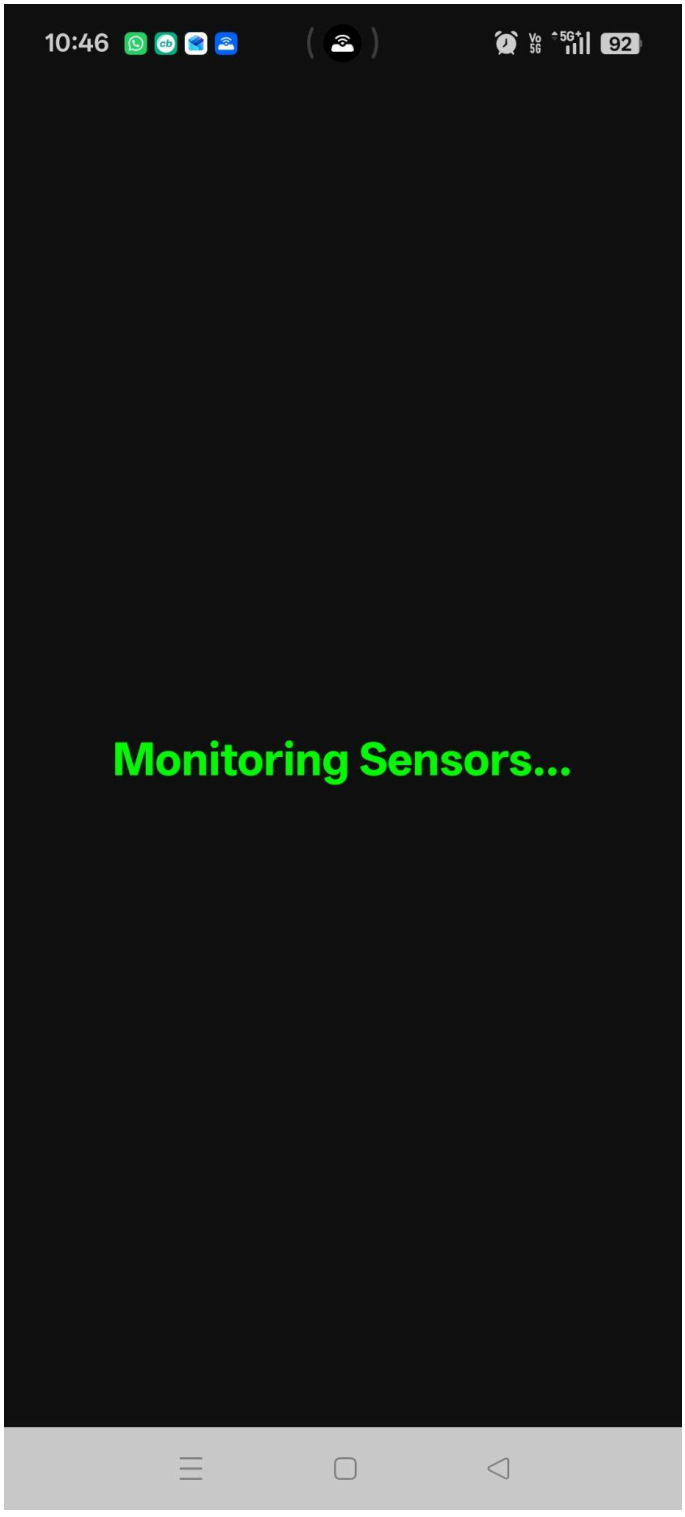
1. **Lack of "High-Impact" ADL Data:** The neural network is essentially a pattern matcher. In the training dataset, the *only* time the Accelerometer recorded a massive, sudden spike in G-force was during a Class 1 Fall. Because the Class 0 (ADL) data only contained smooth movements, the AI learned a dangerous shortcut: *"Any hard physical impact = Fall."*
2. **Dataset Size & Overfitting:** Because the model was trained on a relatively small, single-user dataset, it has not generalized enough to distinguish between a human body hitting a mattress and a plastic phone hitting a wooden table.

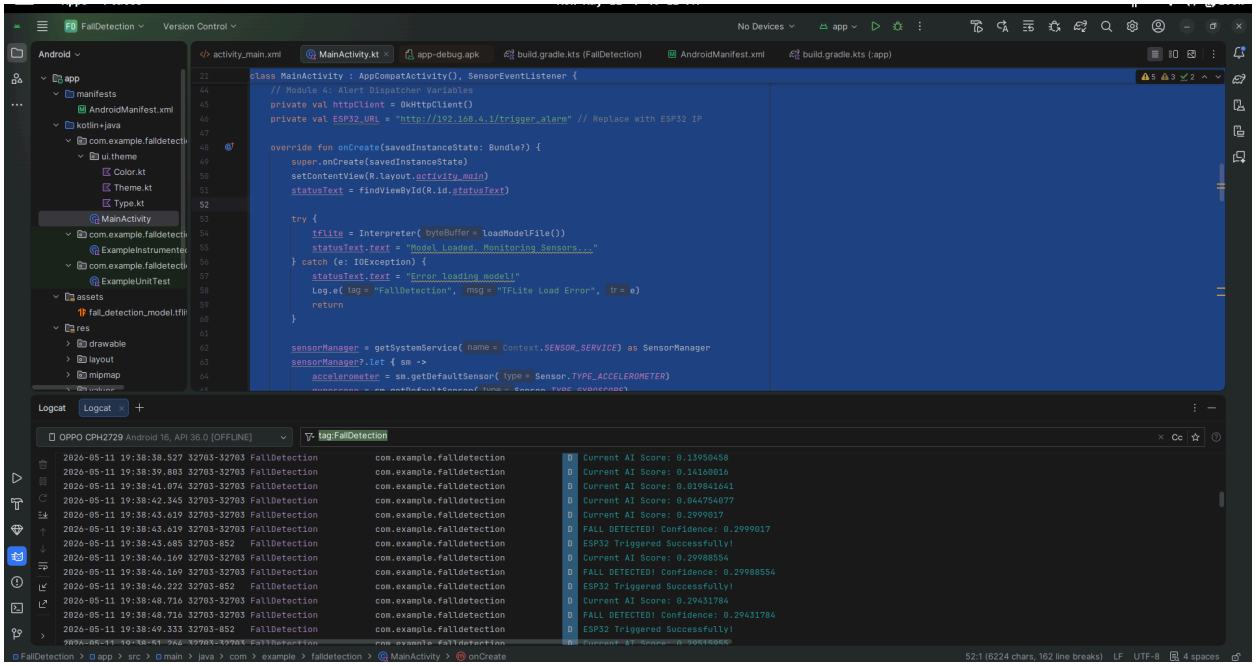
8. Future Improvements

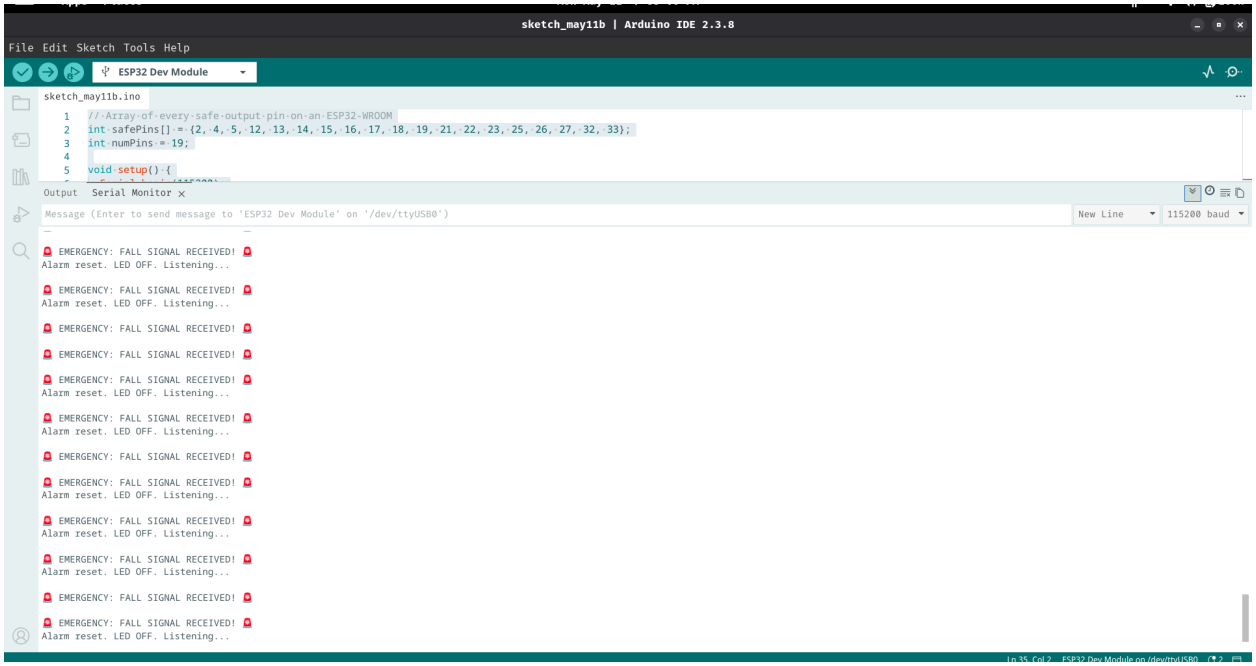
To eliminate false positives and make the model production-ready, the following steps will be taken in V2:

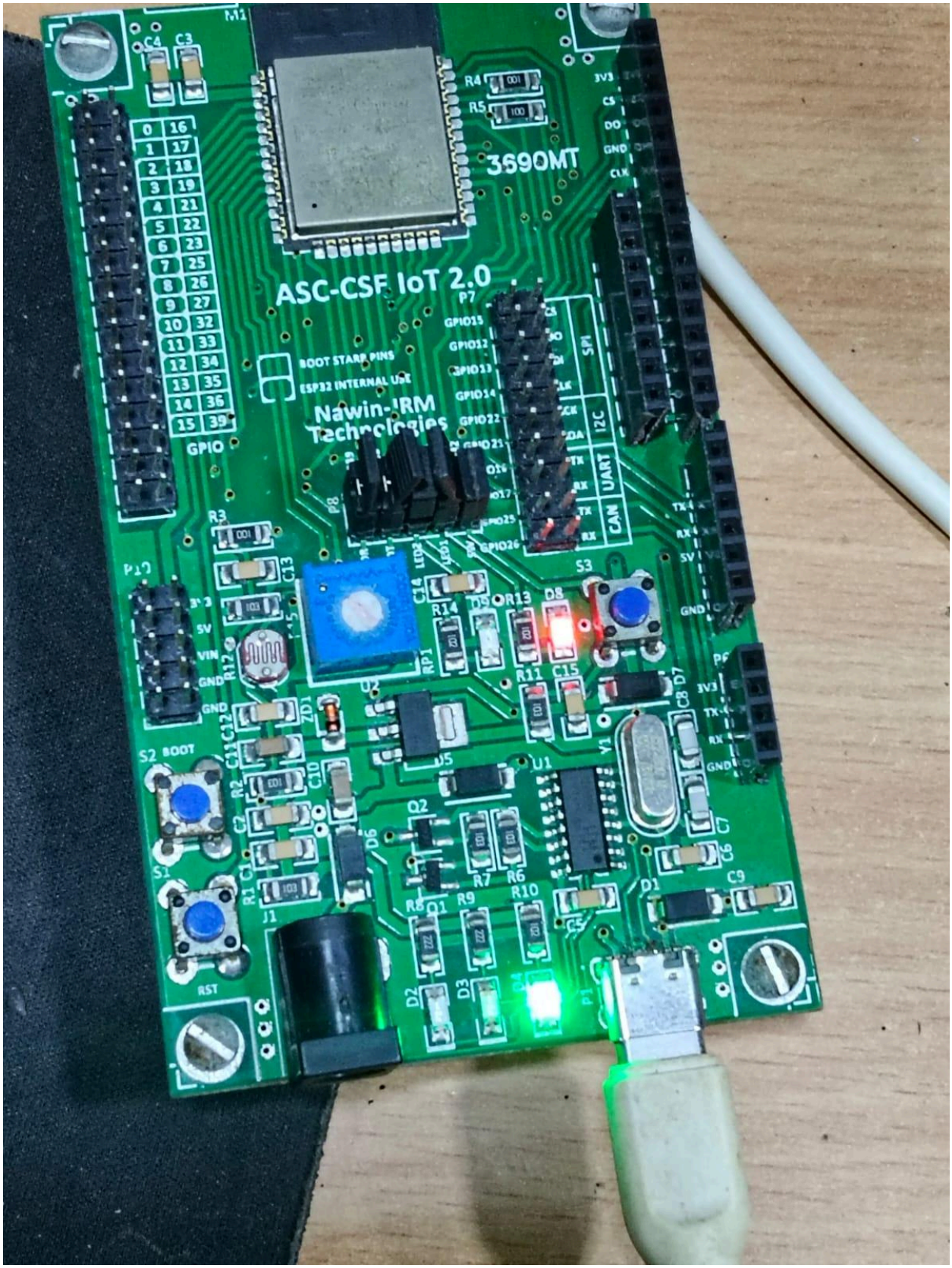
- **Data Augmentation:** Record a new set of Class 0 (ADL) data that specifically includes non-fall high-impact events: dropping the phone on a couch, tossing it onto a table, and running down stairs.
- **Orientation Independence:** Train the model with data collected from multiple pocket orientations (upside down, screen facing outward) so the user does not have to remember a specific phone placement.
- **Hardware Expansion:** Upgrade the ESP32 actuator from an onboard LED to a loud physical buzzer and a GSM module to send SMS alerts to emergency contacts.




10:46
92
Monitoring Sensors...


FallDetection
Version Control
No Devices
app
Android
activity_main.xml
MainActivity.kt
app-debug.apk
build.gradle.kts (FallDetection)
AndroidManifest.xml
build.gradle.kts (:app)
class MainActivity : AppCompatActivity(), SensorEventListener {
// Module 4: Alert Dispatcher Variables
private val httpClient = OkHttpClient()
private val ESP32_URL = "http://192.168.4.1/trigger_alarm" // Replace with ESP32 IP
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_main)
statusText = findViewById(R.id.statusText)
try {
tflite = Interpreter(byteBuffer = loadModelFile())
statusText.text = "Model Loaded. Monitoring Sensors..."
} catch (e: IOException) {
statusText.text = "Error loading model!"
Log.e(tag = "FallDetection", msg = "TFLite Load Error", tr = e)
return
}
sensorManager = getSystemService(name = Context.SENSOR_SERVICE) as SensorManager
sensorManager?.let { sm ->
accelerometer = sm.getDefaultSensor(type = Sensor.TYPE_ACCELEROMETER)
Logcat
OPPO CPH2729 Android 16, API 36.0 [OFFLINE]
tag:FallDetection
2026-05-11 19:38:38.527 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.13950458
2026-05-11 19:38:39.803 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.14160016
2026-05-11 19:38:41.074 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.019841641
2026-05-11 19:38:42.345 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.044754077
2026-05-11 19:38:43.619 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.2999017
2026-05-11 19:38:43.619 32703-32703 FallDetection com.example.falldetection D FALL DETECTED! Confidence: 0.2999017
2026-05-11 19:38:43.685 32703-852 FallDetection com.example.falldetection D ESP32 Triggered Successfully!
2026-05-11 19:38:46.169 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.29988554
2026-05-11 19:38:46.169 32703-32703 FallDetection com.example.falldetection D FALL DETECTED! Confidence: 0.29988554
2026-05-11 19:38:46.222 32703-852 FallDetection com.example.falldetection D ESP32 Triggered Successfully!
2026-05-11 19:38:48.716 32703-32703 FallDetection com.example.falldetection D Current AI Score: 0.29431784
2026-05-11 19:38:48.716 32703-32703 FallDetection com.example.falldetection D FALL DETECTED! Confidence: 0.29431784
2026-05-11 19:38:49.333 32703-852 FallDetection com.example.falldetection D ESP32 Triggered Successfully!
FallDetection › app › src › main › java › com › example › falldetection › MainActivity › onCreate
52:1 (6224 chars, 162 line breaks) LF UTF-8 4 spaces


sketch_may11b | Arduino IDE 2.3.8
File Edit Sketch Tools Help
ESP32 Dev Module
sketch_may11b.ino
//Array of every safe output pin on an ESP32-WROOM
int safePins[] = {2, 4, 5, 12, 13, 14, 15, 16, 17, 18, 19, 21, 22, 23, 25, 26, 27, 32, 33};
int numPins = 19;
void setup() {
Output Serial Monitor
Message (Enter to send message to 'ESP32 Dev Module' on '/dev/ttyUSB0')
New Line
115200 baud
EMERGENCY: FALL SIGNAL RECEIVED!
Alarm reset. LED OFF. Listening...


ASC-CSF IoT 2.0
3690MT
Nawin-IRM
Technologies
BOOT STARP PINS
ESP32 INTERNAL Use
GPIO
P7
P8
P10
P6
SPI
I2C
UART
CAN
S1
S2 BOOT
S3
RST
J1
U1
U2
U5
Q2
Y1
D1
D2
D3
D6
D7
D8
D9
LED1
LED2
R1
R2
R3
R4
R5
R6
R7
R8
R9
R10
R11
R12
R13
R14
RP1
C1
C2
C3
C4
C5
C6
C7
C8
C9
C10
C11
C12
C13
C14
C15
ZD1
3V3
5V
VIN
GND
CS
DO
CLK
TX
RX